

Project Group: *BM4*

Group Details:

Name	University ID
<i>A.D.T.M.S.H. Tennekone</i>	<i>240642J</i>
<i>U.I.D. Thennakoon</i>	<i>240657J</i>
<i>W.M.I.R. Wanninayaka</i>	<i>240695X</i>

Project Title:

Smart Water Tank Monitoring and Management System

Abstract:

- Problem – Unmonitored water tanks
- Objective – Building a smart monitoring system
- Solution – A Raspberry Pi system using ultrasonic, pH and flow sensor
- Key Outcomes – Successful notifications via Blynk IoT

Problem Addressed

1. *Risk of Overflow or Shortage:*

- In traditional systems, there is no automatic mechanism to give a notification to user to stop the water pump when the tank reaches capacity. This frequently leads to tank overflows that leads go a significant wastage of treaded water. Additionally, if we use electric water pump, the unnecessary operation of the water pump wastes electricity and increasing household utility costs.
- Without real-time monitoring, users often only realize the tank is empty when the water supply ceases at the tap. At that user require an emergency wait time for the tank to refill.

2. *Inconvenience and Safety:*

- The only way to verify the water level in a standard system is through physical inspection (Ex: Climbing a ladder to check the tank). This is inconvenient, time-consuming, and poses a safety risk, particularly during bad weather or for elderly users.

3. *Uncertain Water Quality:*

- Clear water is not necessarily safe water. Stagnant water in storage tanks is an ideal breeding ground for invisible pathogens like bacteria, viruses and protozoa, which can't be detected by visual inspection alone.
- Over time, sediment from the supply line settles at the bottom of the tank. This creates a sludge layer that encourages the growth of biofilm such as a slimy, bacteria-rich coating on the tank walls that can re-contaminate fresh water entering the tank.
- Environmental factors like acid rain or degrading plastic piping can alter the water's pH. Consuming water that is too acidic (low pH) can strip metals from plumbing, leading to heavy metal poisoning, while highly alkaline water (high pH) tastes bitter and reduces the effectiveness of chlorine disinfection.
- The consumption of unmonitored, contaminated tank water is leading cause of waterborne diseases such as typhoid, cholera and dysentery.
- Currently, users rely on periodic manual cleaning, which is often reactive which performed only after the water has already become visibly dirty or foul-smelling.

4. *Lack of Consumption Data:*

- In traditional systems, water meters are only read monthly. Users don't have way to know their daily consumption habits until they receive a high bill at the end of the month. There is no real-time feedback loop to warn them of excessive usage.

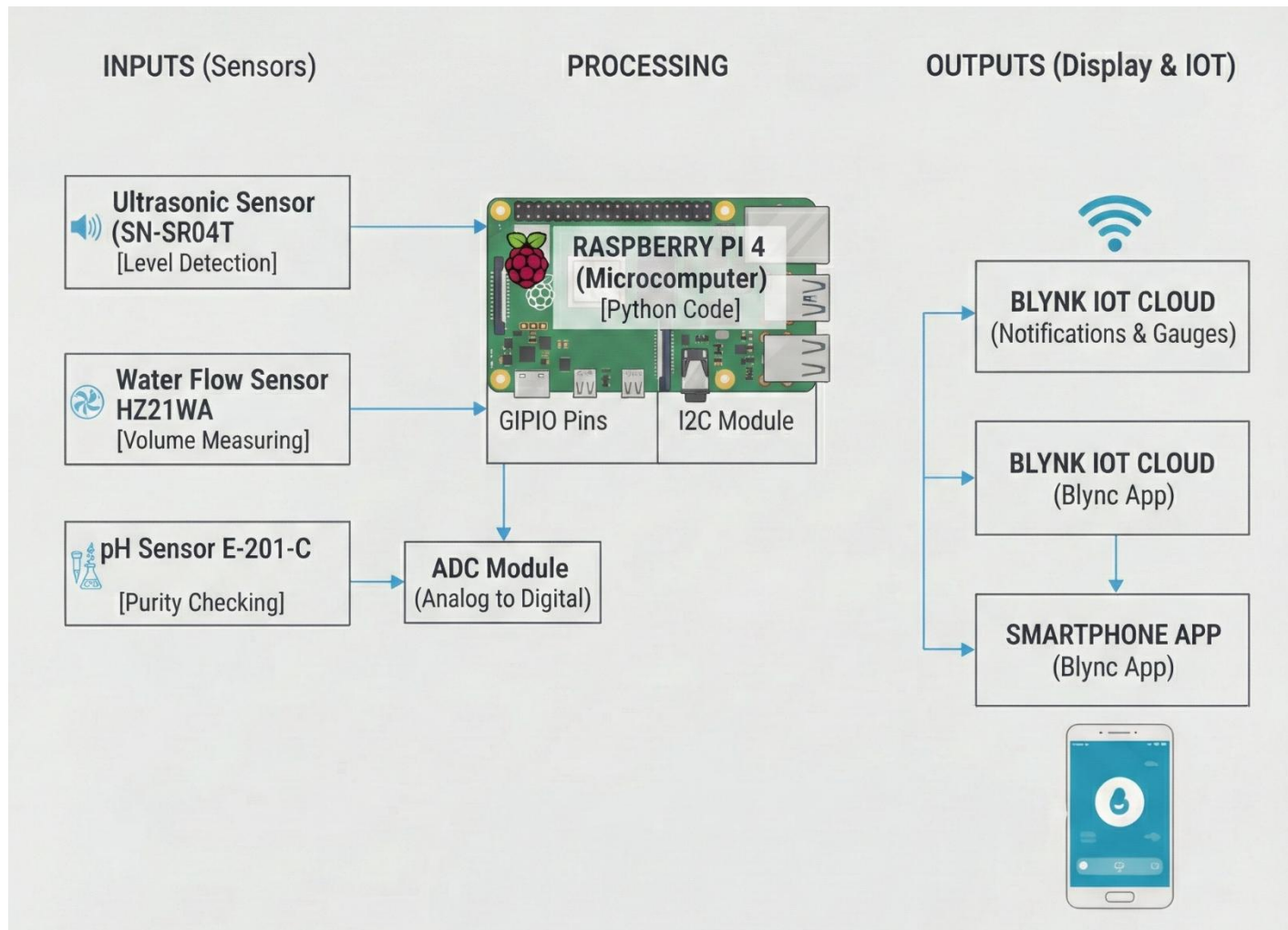
Project Goal

- Design and implement a low-cost, IoT based 'Smart Water Tank Monitoring and Management System'.

Specific Objectives

1. To measure the tank's water level in real-time using an ultrasonic sensor.
2. To assess water purity by measuring its pH level using pH sensor.
3. To measure the volume of water consumed using a water flow sensor.
4. To display the consumed water volume locally on a LCD display.
5. To send all sensor data and critical alerts to the user's smartphone via Blynk IoT platform.

System Architecture



Hardware Components

- Raspberry Pi 4 Model B
- JSN – SR04T Ultrasonic Sensor
- E-201-C pH sensor
- HZ21WA Flow Sensor
- ADS1115 Analog-to-Digital Converter (16 bit)
- Dot Board
- Circuit wire, Jumper wires
- LEDs, Busser, Resistors

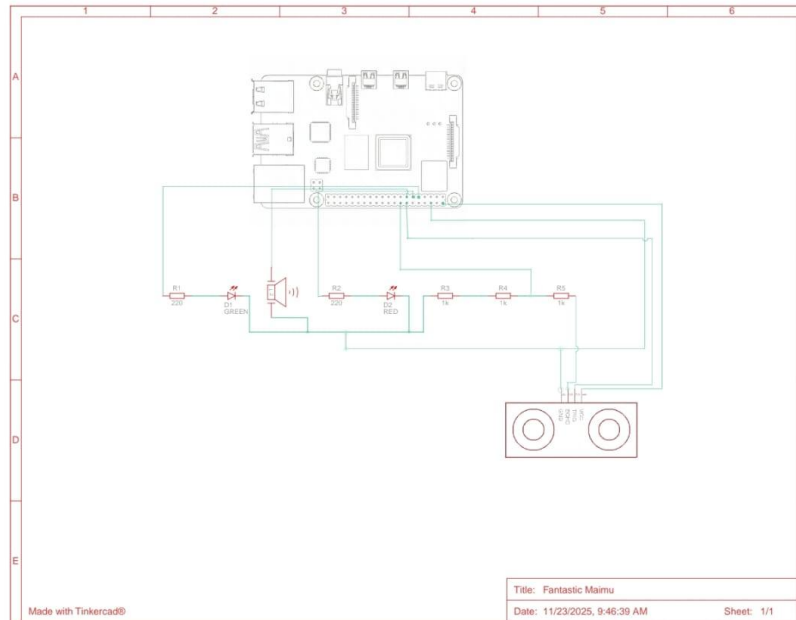
Software Design

- Programming Language: Python 3
- Key Libraries:
 - RPi.GPIO
 - time
 - BlynkLib
 - Statistics
 - board, busio
 - adafruit_ads1x15.ads1115
 - adafruit_ads1x15.analog_in

Sub-system Implementation

1. Tank water level system:

- **Circuit Diagram:**



- **Principle of Operation:**

- The JSN-SR04T sensor measures distance using the 'Time-of-Flight' principal.
 1. *Tigger:* The Raspberry Pi sends a 10 μs HIGH pulse to the sensor's TRIG pin.
 2. *Brust:* The sensor emits a burst of eight ultrasonic pulses at 40 kHz. These sound waves travel through the air inside the tank towards the water surface.
 3. *Reflection:* When the sound waves hit the water surface, they are reflected back towards the sensor.
 4. *Reception:* The sensor detects the returning echo and sets the ECHO pin to HIGH for a duration proportional to the time the sound took to travel.
- **Distance Calculation Formula:**
 1. $Distance (cm) = \frac{Speed\ of\ Sound \times Time\ Elapsed}{2}$
- **Converting Distance to Water Level:**
 1. The ultrasonic sensor measures the Air Gap. So, we apply the following logic in Python code.

- Calibration: We first measure the Total Tank Depth.
- Subtraction: The sensor provides the current distance to the water surface. The actual height of the water is calculated by subtracting the sensor reading from the total depth.

✓ **Water Level =**

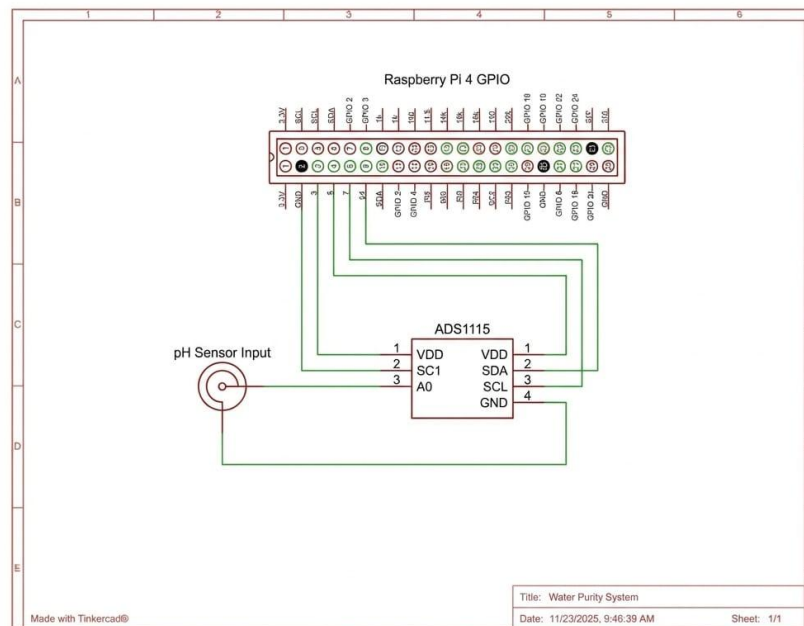
Total Tank Depth – Sensor Reading

○ **Percentage Calculation:**

$$1. \text{ Tank Percentage} = \left(\frac{\text{Water Level}}{\text{Total Tank Depth}} \right) \times 100$$

2. Water Purity (pH) System:

- **Circuit Diagram:**



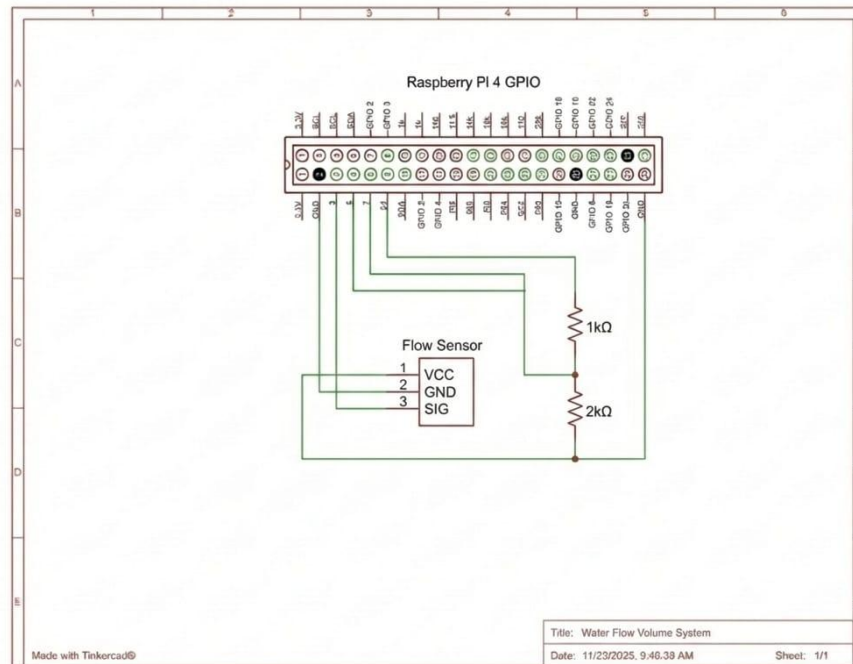
- **Principle of Operation:**

- The pH sensor operates like a small battery. It generates a minute electrical potential (voltage) based on the concentration of Hydrogen ions in the water. This output is an Analog Signal, meaning it varies continuously and can be any value.
- The Raspberry Pi 4 is a purely Digital device. Its GPIO pins can only detect two states: HIGH (3.3V) or LOW (0V). It lacks internal hardware to read varying analog voltage.

- To bridge this gap, an ADS1115 Analog-To-Digital Converter is used. The ADS1115 samples the continuous voltage from the pH sensor and converts it into precise 16-bit digital number. This digital data is then transmitted to the Raspberry Pi via the I2C communication protocol, allowing the Python script to process the pH data.
- Calibration Process:
 - pH electrodes are sensitive electrochemical devices that drift over time due to electrode aging and temperature changes. Therefore, raw voltage can't be directly translated to pH without calibration. We utilized a Two-Point Calibration method to determine the linear relationship between voltage and pH.
 1. Standard Buffer Solutions:
 - We used two reference solutions with known pH values
 2. Data Collection:
 - The sensor was placed in the pH 4.01 solution, and the average voltage (V1) was recorded.
 - The sensor was cleaned and placed in the pH 9.18 solution, and the average voltage (V2) was recorded.
 3. Linear Regression:
 - Assuming a linear response, we calculate the Slope and Intercept for equation $y = mx + c$, where y is the pH and x is the Voltage.
 - ✓ $Slope = \frac{pH2 - pH1}{V2 - V1}$
 - ✓ $Intercept = pH1 - (Slope \times V1)$
 4. Code Implementation:
 - These calculate constants were entered into the Python script as `SLOPE = -10.87` and `INTERCEPT = 36.63`. The system uses these values to convert real-time voltage readings into accurate pH levels using the formula
 - ✓ $Current\ pH = (Slope \times Measured\ Voltage) + Intercept$

3. Water Consumption System:

- **Circuit Diagram:**



- **Principle of Operation:**

- To measure water usage, the system utilizes an HZ21WA water flow sensor. This component operates on the principal of Hall Effect, which states that a voltage difference is generated across an electrical conductor when it is placed in a magnetic field

1. Mechanical Action

- Inside the sensor body is a plastic turbine equipped with a small magnet. When water flows through the sensor, it pushes against the turbine blades, causing the rotor to spin.

2. Magnetic Detection:

- A Hall Effect sensor chip is embedded in the static casing of the device, sitting just above the rotor.

3. Pulse Generation:

- As the rotor spins, the magnet passes the Hall sensor with every revolution. Each time the magnetic field passes the sensor, it toggles the output pin from LOW (0V) to HIGH (5V) and back. This creates a stream of digital square wave pulses.

4. Relation to flow:

- The speed of the rotor is directly proportional to the speed of water flow. Therefore, a higher flow rate results in a higher flow rate results in a higher frequency of pulse.

- Data Processing and Calculation:

- Instead of constantly checking the pin state, which wastes CPU resources, the code uses Hardware Interrupts. The GPIO pin is configured to detect a 'Rising Edge' (the moment the signal jumps from 0 to 1). A counter variable is incremented automatically every time a pulse is detected.
- The flow rate is calculated by measuring the frequency of these pulses (Hz). We use a specific Calibration Factor found in the sensor's datasheet. The formula used is:

- $$\text{Flow Rate } \left(\frac{L}{\text{min}} \right) = \frac{\text{Pulse Frequency (Hz)}}{\text{Calibration Facotr (K)}}$$

- Calculating Total Volume:

- $$\text{Volume Increment} = \frac{\text{Flow rate}}{60} \times \Delta t \text{ (seconds)}$$

- $$\text{Total Volume} = \sum(\text{Volme Increment})$$

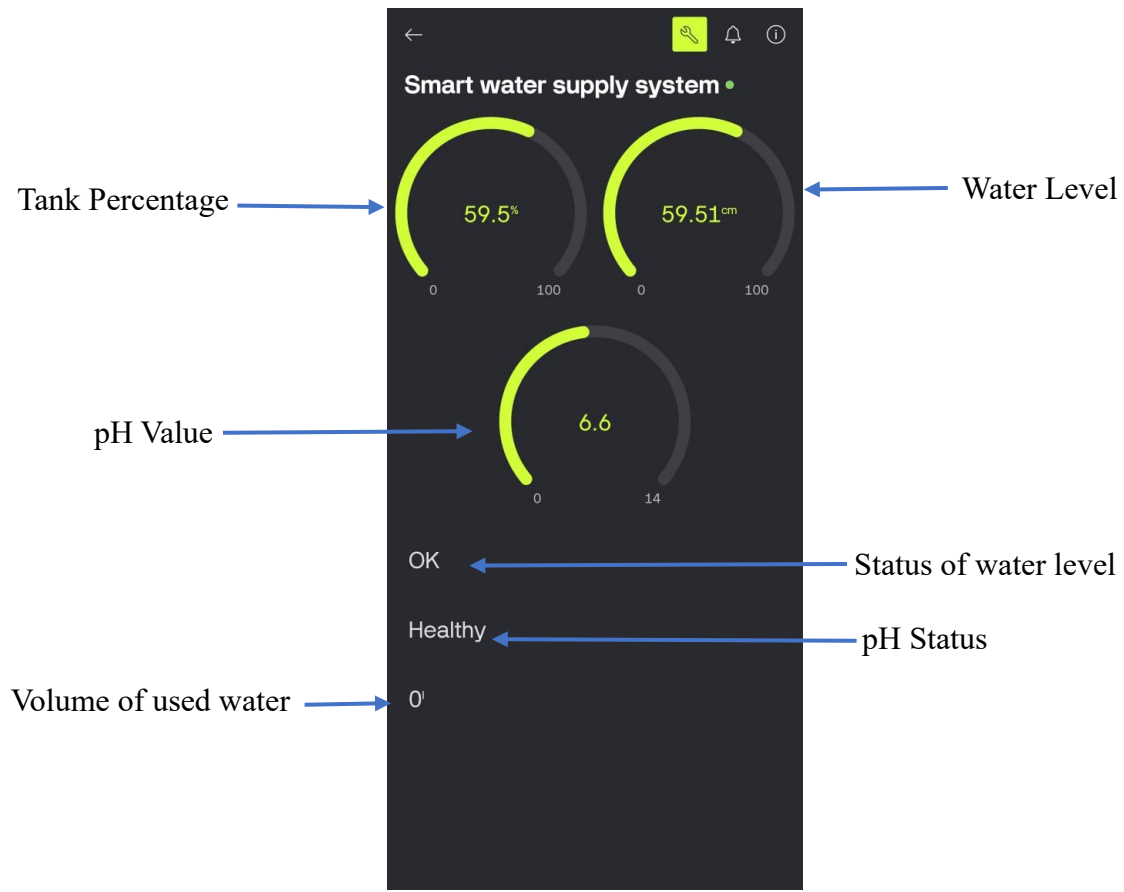
4. Blynk Integration

- To achieve the project requirement of 'Remote monitoring', the system integrates with Blynk, a comprehensive IoT platform. This interface bridges the gap between the physical hardware and the user. The system utilizes the BlynkLib Python library to establish a secure, bi-directional connection via the standard TCP/IP protocol, using a unique Authentication Token to identify the specific device.
- In the Blynk architecture, data is not sent to physical pins but to 'Virtual Pins'. These act as data channels linking the Python code to specific widgets on the mobile dashboard. The complete mapping for full system is defined as follows:

-

Virtual Pin	Data Point	Widget Type Used	Description
V0	Status of water level	Text Display	Shows 'OK', 'Tank Full'
V1	Water level (cm)	Gauge	Shows the raw distance to water
V2	Tank Percentage (%)	Gauge	Visualize how full the tank is (0-100%)
V3	pH value	Gauge	Displays acidity level (0-14)
V4	pH status	Text Display	Shows 'Healthy' of 'Unhealthy' based on range
V5	Volume of used water (l)	Text Display	Shows the used volume of water

- **Mobile Dashboard:**



- **Data Transmission Logic:**

- The `blynk.run()` command is executed continuously to maintain server connection.
- Unlike traditional systems where the users must 'request' data, our system utilizes a 'Push' architecture. The Raspberry Pi proactively sends data every 5 seconds.
- Alert Automation:
 - In-App Automation: We utilized the Blynk console's 'Automations' feature. Logic was set up on the server side.
 - Result: When the tank level drops critically low, the user receives a standard Push Notification on their phone lock screen, ensuring they are alerted even if app is closed.

Ideas for Future Improvements

1. Enhanced Water Quality Parameters

- Currently, the system relies solely on pH to assess water quality. However, safe drinking water requires monitoring multiple parameters.
 - **TDS Sensor Integration:**
 - Adding a TDS sensor would allow the system to detect the concentration of dissolved particles (mineral, salts, metals). High TDS often indicates contaminated water.
 - **Turbidity Sensor:**
 - Incorporating a Turbidity sensor would measure the cloudiness of the water, allowing the system to detect suspended particles, mud, or algae growth, which a pH sensor cannot detect.

2. Closed-Loop Automation

- The current system acts as a monitoring device, alerting the user to take action. A future upgrade would convert this into a control system.
 - Relay Module: By connecting a 5V Relay Module to the Raspberry Pi, the system could automatically switch the water pump ON when the water level is low and OFF when it reaches 90%. This removes the need for human intervention entirely.

3. Machine Learning for Leak Detection

- Currently, leak detection relies on basic threshold logic.
 - Usage Pattern Analysis: By implementing simple Machine Learning model on Raspberry Pi, the system could learn the household's normal usage patterns.
 - Anomaly Detection: If the flow sensor detects water usage unusual time or a constant low flow rate for an extended period, the AI would identify this as a Leak and trigger an emergency alert, distinguishing it from normal consumption.

References

- Level: <https://www.youtube.com/watch?v=gOwGwB9fZBo> (Water Level Indicator using Ultrasonic Sensor and Raspberry Pi)
- Flow: <https://www.youtube.com/watch?v=wpenAP8gN3c> (RPi 22 - YF-S201 Water Flow Meter/Sensor, Interrupt based - ADVANCED)
- ADC/pH: <https://www.youtube.com/watch?v=wJgyszOSoQU> (Raspberry Pi Analog Water Sensors ADC Tutorial)
- ChatGPT AI Chatbot
- Gemini AI Chatbot

